Multinomial Naive Bayes

It is one of the variation of Naive Bayes algorithm which is idea for discrete data and is typically used in text classification problems based on word frequencies like in spam email detection.

The word '**Naive**' assumes that all features like words in a sentence are independent from each other. And '**Multinomial**' refers to how many times a word appear or how often a category occurs.

Probability Density Function:

$$P(X) = \frac{n!}{n_1!n_2!\ldots n_m!} p_1^{n_1} p_2^{n_2} \ldots p_m^{n_m}$$

Where

- n is the total number of trials
- n_i is the count of occurrences for outcome i
- p_i is the probability of outcome i

In order to estimate how likely each word is in a particular class like "spam" or "not spam" we use a method called Maximum Likelihood Estimation (MLE). this help finding probabilities based on actual counts from our data.

$$\theta_{c.i} = \frac{\text{count}(w_i, c) + 1}{N + v}$$

Where

- $count(wi, c)$ is the number of times word wiwi appears in documents of class c.
- N is the total number of words in documents of class c.
- v is the vocabulary size.

Example

To understand how Multinomial Naive Bayes works, here's a simple example to classify whether a message is **"spam"** or **"not spam"** based on the presence of certain words.

Message ID	Message Text	Class
M1	"buy cheap now"	Spam
M2	"limited offer buy"	Spam
M3	"meet me now"	Not Spam
M4	"let's catch up"	Not Spam

1. Vocabulary

Extract all unique words from the training data:

$$\text{Vocabulary} = \{\text{buy, cheap, now, limited, offer, meet, me, let's, catch, up}\}$$

Vocabulary size $V = 10$

2. Word Frequencies by Class

Spam Class (M1, M2):

- buy: 2
- cheap: 1
- now: 1
- limited: 1
- offer: 1

Total words: 6

Not Spam Class (M3, M4):

- meet: 1
- me: 1
- now: 1
- let's: 1
- catch: 1
- up: 1

Total words: 6

3. Test Message

Test Message: "buy now"

4. Applying Multinomial Naive Bayes Formula

$$P(C|d) \propto P(C) \cdot \prod_i P(w_i|C)^{f_i}$$

Prior Probabilities:

$$P(\text{Spam}) = 0.5, \quad P(\text{Not Spam}) = 0.5$$

Apply Laplace Smoothing:

$$P(w \mid C) = \frac{\text{count}(w,C)+1}{\text{total words in } C+V}$$

Spam Class:

- $P(\text{buy} \mid \text{Spam}) = \frac{2+1}{6+10} = \frac{3}{16}$
- $P(\text{now} \mid \text{Spam}) = \frac{1+1}{6+10} = \frac{2}{16}$

$$P(\text{Spam} \mid d) \propto 0.5 \cdot \frac{3}{16} \cdot \frac{2}{16} = \frac{3}{256}$$

Not Spam Class:

- $P(\text{buy} \mid \text{Not Spam}) = \frac{0+1}{6+10} = \frac{1}{16}$
- *P(\text{now} \mid \text{Not Spam}) = \frac{1 + 1}{6 + 10} = \frac{2}{16}*

$$P(\text{Not Spam} \mid d) \propto 0.5 \cdot \frac{1}{16} \cdot \frac{2}{16} = \frac{1}{256}$$

5. Final Classification

Since, $P(\text{Spam}|d) = \frac{3}{256} > \frac{1}{256} = P(\text{Not Spam}|d)$

$\boxed{\text{The message is classified as Spam}}$

Example from Geek4Geek

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

data = {
    'text': [
        'Free money now',
        'Call now to claim your prize',
        'Meet me at the park',
        'Let’s catch up later',
        'Win a new car today!',
        'Lunch plans?',
```

```
        'Congratulations! You won a lottery',
        'Can you send me the report?',
        'Exclusive offer for you',
        'Are you coming to the meeting?'
    ],
    'label': ['spam', 'spam', 'not spam', 'not spam', 'spam', 'not spam', 'spam',
'not spam', 'spam', 'not spam']
}

df = pd.DataFrame(data)

df['label'] = df['label'].map({'spam': 1, 'not spam': 0})

X = df['text']
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

X = df['text']
y = df['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

vectorizer = CountVectorizer()
X_train_vectors = vectorizer.fit_transform(X_train)
X_test_vectors = vectorizer.transform(X_test)

model = MultinomialNB()
model.fit(X_train_vectors, y_train)

y_pred = model.predict(X_test_vectors)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%\n")

custom_message = ["Congratulations, you've won a free vacation"]
print(custom_message)
custom_vector = vectorizer.transform(custom_message)
prediction = model.predict(custom_vector)
print("Prediction for custom message:", "Spam" if prediction[0] == 1 else "Not
Spam")
```

From Geek4Geek